REBOUND

A PROJECT REPORT

Submitted by

AKSHITA SHARMA [RA2211026010268]
ANSHITA SRIVASTAVA [RA2211026010278]

Under the Guidance of

DR. R. SIVA

(Associate Professor, Department of Computational Intelligence)

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Artificial Intelligence and Machine Learning



DEPARTMENT OF COMPUTATIONAL INTELLIGENCE
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE ANDTECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026

Department of Computational Intelligence
SRM Institute of Science & Technology
Own Work Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

Degree/ Course : BTech in Computer Science Engineering with Specialization in Artificial Intelligence and Machine Learning

Student Name : Akshita Sharma, Anshita Srivastava

Registration Number :RA2211026010268, RA2211026010278

Title of Work : Rebound

I / We hereby certify that this assessment compiles with the University's Rules and Regulations relating to academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

I / We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook /University website

I / We understand that any false claim for this work will be penalized in accordance with theUniversity policies and regulations.

DECLARATION:
I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above. Akshita Sharma [RA2211026010268] Anshita Srivastava [RA2211026010278]

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP401L/21CSP402L – Major Project [21CSP403L-Internship] report titled "**REBOUND**" is the bonafide work of "**AKSHITA SHARMA[RA2211026010268], ANSHITA SRIVASTAVA[RA2211026010278]**" who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

DR. R. SIVA

SUPERVISOR
ASSOCIATE PROFESSOR
DEPARTMENT OF
COMPUTATIONAL
INTELLIGENCE

SIGNATURE

DR. R. ANNIE UTHRA

HEAD OF DEPARTMENT
DEPARTMENT OF
COMPUTATIONAL INTELLIGENCE

EXAMINER 1

EXAMINER 2

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. C Lakshmi,** Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. R. Annie Uthra,** Department of Computational Intelligence, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor **Dr. Nagendra Prabhu S**, Department of Computational Intelligence, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. R. Siva** Department of Computational Intelligence, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his mentorship. He provided us with the freedom and support to explore the research topics of our interest. His passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Department of Computational Intelligence, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Authors

ABSTRACT

Systematic performance analysis in sports requires structured and accessible performance data. However, amateur athletes are not offered suitable means of efficiently recording their game statistics. This means that their performance is not well documented. The study seeks to create a centralized game logging system that is effective in analyzing performance data.

The system is intended to offer a unified platform that can be used in documenting game events, recording statistics, and retrieving analysis based on aggregated data. The system is based on a mobile platform that ensures performance tracking is consistent. The study's findings highlight the need for centralized game logging in performance documentation and analysis. By integrating a mobile interface with a cloud-based data infrastructure, the platform supports consistent performance tracking and improved accessibility of sports data. The results demonstrate the centralized game logging can enhance performance documentation and provide meaningful analytical insights for athletes and coaches.

TABLE OF CONTENTS

ABSTRACT		**v**
TABLE OF CONTENTS		**vi**
LIST OF FIGURES		**vii**
LIST OF TABLES		**viii**
ABBREVIATIONS		**ix**
CHAPTER NO.	**TITLE**	**PAGE NO.**
1	**INTRODUCTION**	**1**
	1.1 Introduction to Project	2
	1.2 Problem Statement	3
	1.3 Motivation	4
	1.4 Sustainable Development Goal of the Project	5
2	**LITERATURE SURVEY**	**6**
	2.1 Overview of the Research Area	7
	2.2 Existing Models and Frameworks	8
	2.3 Limitations Identified from Literature Survey (Research Gaps)	11
	2.4 Research Objectives	12
	2.5 Product Backlog (Key user stories with Desired outcomes)	13
	2.5 Plan of Action (Project Road Map)	15
3	**SPRINT PLANNING AND EXECTION METHODOLOGY**	**21**
	3.1 SPRINT I **User Onboarding and Team Establishment**	21
	3.1.1 Objectives with user stories of Sprint I	23
	3.1.2 Functional Document	25
	3.1.3 Architecture Document	28
	3.1.4 Outcome of objectives/ Result Analysis	29
	3.1.5 Sprint Retrospective	30
	3.2 SPRINT II **Game Logging and Performance Analytics**	31
	3.2.1 Objectives with user stories of Sprint II	32
	3.2.2 Functional Document	33
	3.2.3 Architecture Document	34
	3.2.4 Outcome of objectives/ Result Analysis	35
	3.2.5 Sprint Retrospective	36

3.3 SPRINT III **Administrative Control and System Refinement** 37
3.3.1 Objectives with user stories of Sprint III 38
3.3.2 Functional Document 39
3.3.3 Architecture Document 40
3.3.4 Outcome of objectives/ Result Analysis 41
3.3.5 Sprint Retrospective 42
4 RESULTS AND DISCUSSIONS 40
4.1 Performance Evaluation
4.2 Model Comparison
4.3 Visualization and Analysis
4.4 Discussion of Results

5 CONCLUSION AND FUTURE ENHANCEMENT 45
REFERENCES 46
APPENDIX
A CODING 48
B CONFERENCE PUBLICATION 50
C JOURNAL PUBLICATION 51
D PLAGIARISM REPORT 52

LIST OF FIGURES

CHAPTER NO.	TITLE	PAGE NO.
1	Athlete Management Workflow	1
2	Team-Level Analysis	3
3	Performance Analysis System	4
4		
5		

LIST OF TABLES

CHAPTER	TITLE	PAGE
1	Detailed User Stories of Sprint	1
2	Access Level Authorization Matrix SP-1	2
3	Functional Test Cases for Sprint 1	3
4	Detailed User Stories of Sprint 1	4
5	Access Level Authorization Matrix SP-2	5
6	Functional Test Cases for Sprint 2	6
7	Detailed User Stories of Sprint 1	7
8	Access Level Authorization Matrix SP-3	8
9	Functional Test Cases for Sprint 3	9

ABBREVIATIONS

- 2P: Two Points Scored

• 3P: Three Points Scored

• STL: Steals

• PTS: Points

• Position Abbreviations:

  o PG: Point Guard

  o SG: Shooting Guard

  o SF: Small Forward

  o PF: Power Forward

  o C: Center

  o G: Guard

  o F: Forward

• PPG: Points per Game

• RPG: Rebounds per Game

• AST: Assists • PFG: Point Field Goals

• FT: Free Throw(s)

CHAPTER 1

INTRODUCTION

1.1 Introduction to Rebound

In the world of sports, consistent performance tracking and documentation are essential for growth, recognition, and opportunity. However, most existing platforms cater primarily to professional athletes or high-profile teams, leaving a significant gap for aspiring athletes and local teams who lack access to organized tools for tracking their sports journey. This is the problem Rebound aims to solve.

Rebound is a web application developed to provide athletes and coaches with a centralized platform to record, analyse, and reflect on their game performances. Whether it's logging the results of a local basketball match, tracking individual player statistics like assists and rebounds, or sharing milestones through posts, Rebound ensures that every athlete—regardless of level—has access to professional-grade tools for their development.

The project introduces features such as game and team creation, stat-based game logs, timeline visualizations, and searchable athlete/coach profiles. By integrating performance data with social sharing and portfolio-building capabilities, Rebound allows athletes not only to improve but to showcase their progress and attract opportunities like sponsorships, team selections, and scouting attention.

In essence, Rebound is more than just a stats tracker—it is a digital companion for athletes and coaches who want to take ownership of their sports journey in a structured and impactful way.

1.2 Problem Statement

In today’s sports ecosystem, aspiring athletes and local teams often face significant challenges in documenting, analysing, and showcasing their performance. Unlike professional athletes who have access to advanced analytics tools, scouting platforms, and media exposure, amateur athletes frequently rely on manual record-keeping methods such as memory, notebooks, spreadsheets, or informal communication to track their progress. This lack of a centralized and structured system makes it difficult for athletes to monitor performance trends, identify areas for improvement, and build credible portfolios that can help them gain recognition from coaches, scouts, sponsors, and recruiters.

Coaches and team managers also encounter difficulties in maintaining consistent player records, managing teams, and analysing game statistics over time. Existing sports platforms primarily focus on professional leagues or elite athletes, leaving grassroots players, school teams, college athletes, and independent clubs underserved. Additionally, there is limited integration between performance tracking, team management, social sharing, and portfolio-building tools within a single platform.

Rebound addresses this problem by providing a centralized, user-friendly platform where athletes and coaches can log games, track player statistics, analyse performance through graphical insights, manage teams, and showcase achievements. By combining innovation, data analytics, and community-driven features, Rebound aims to bridge the gap between amateur and professional sports documentation, empowering athletes with better opportunities for growth, recognition, and career advancement.

Table 1: Advantages of Proposed System

Feature	Description	Benefit
Centralized Data	Stores all athlete data in one place	Easy access and management
Performance Tracking	Tracks player statistics	Better decision-making
Analytics Dashboard	Visual representation of data	Improved insights
Real-time Updates	Updates match data instantly	Accurate information
User-Friendly Interface	Simple UI for users	Easy usability

1.3 Motivation

The idea behind Rebound was born from a real and recurring challenge faced by local athletes and coaches—there is no accessible, organized way to track and analyze athletic performance at the grassroots or amateur level. While professional athletes benefit from advanced analytics, scouting platforms, and media exposure, aspiring players often rely on memory, scattered notes, or informal word-of-mouth to document their achievements and growth. This lack of structure not only hinders self-improvement but also creates barriers when athletes seek recognition, sponsorships, or career advancement. Coaches, too, struggle to evaluate performance trends over time without consistent records. Motivated by this gap, Rebound seeks to democratize access to performance tracking tools by offering an intuitive, affordable, and centralized mobile platform. The goal is to empower every athlete—from school-level players to independent club members—with the ability to log games, visualize progress, share accomplishments, and build a credible sports portfolio.

The project is driven by a passion to bridge the inequality in sports analytics and provide every athlete with the tools they need to grow, be discovered, and reach their full potential.

1.4 Sustainable Development Goal of the Project

The development of Rebound aligns with the United Nations' Sustainable Development Goal 9: Industry, Innovation, and Infrastructure, which focuses on building resilient infrastructure, promoting inclusive and sustainable industrialization, and fostering innovation. Rebound contributes to this goal by introducing digital innovation into the grassroots sports sector—a field often overlooked by mainstream technology solutions. By leveraging mobile technology and data-driven insights, Rebound creates a scalable digital infrastructure that supports athletes and coaches in documenting, analyzing, and enhancing sports performance. The platform promotes inclusivity by offering structured tools that are accessible to athletes beyond elite institutions—those from local communities, schools, and underrepresented teams. It fosters innovation by redefining how athletic progress is tracked and communicated, replacing outdated manual methods with real-time, visual analytics and performance logs. Ultimately, Rebound represents a forward-thinking approach to modernizing the sports ecosystem at every level, contributing to the broader SDG goal of building innovation-driven, equitable digital infrastructure for all.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview of the Research Area

The integration of digital technologies in sports has transformed how performance is analysed, recorded, and utilized for decision-making. Sports performance analytics systems focus on collecting, storing, and analysing data related to player activities, match statistics, and team performance. These systems help athletes and coaches track progress, identify strengths and weaknesses, and improve overall performance through data-driven insights.

Despite advancements in sports technology, most solutions are designed for professional environments and require access to sophisticated infrastructure. At the amateur and semi-professional levels, athletes often rely on informal methods such as manual record-keeping or basic applications that lack structure and reliability. There is a growing need for platforms that provide structured data management, performance tracking, and verification mechanisms while remaining accessible and easy to use.

The research area, therefore, focuses on developing scalable and user-friendly systems that combine performance documentation, analytics, team management, and social interaction into a unified digital platform.

2.2 Existing Models and Frameworks

Various systems and platforms exist that partially address sports performance tracking and athlete management:

- **Professional Sports Analytics Systems**
  These platforms provide advanced analytics, real-time data processing, and detailed performance insights. However, they are costly, complex, and primarily designed for elite-level sports organizations.
- **Fitness Tracking Applications**
  General-purpose applications allow users to monitor health metrics and physical activities. While useful for individual tracking, they do not support structured match data, team workflows, or sport-specific analytics.
- **Team and Club Management Platforms**
  These systems focus on team coordination, scheduling, and communication. However, they lack detailed performance logging, validation workflows, and analytical visualization features.

- **Standalone Game Logging Tools**
  Some tools enable users to manually record match statistics. These systems often lack verification processes, role-based approvals, and advanced analytics.
- **Social Media Platforms**
  Athletes frequently use social platforms to showcase achievements. However, these platforms do not provide structured, authenticated, or data-driven performance records.

Overall, existing frameworks address isolated aspects of sports management but fail to provide an integrated, end-to-end solution.

2.3 Limitations Identified from Literature Survey (Research Gaps)

Based on the study of existing systems and practical workflows, the following research gaps are identified:

- Absence of **centralized platforms** combining performance tracking, validation, and showcasing
- Lack of **role-based workflows** (athlete, coach, admin) for managing and approving data
- No **verification mechanism** for ensuring authenticity of match statistics
- Limited support for **team-based collaboration and management**
- Inadequate **performance visualization tools** such as timelines and dashboards
- Lack of **structured onboarding and profile management systems**
- Absence of **approval systems** (e.g., coach approval for game logs, admin approval for coaches)
- Poor integration between **analytics and social sharing features**
- Existing systems are either **too complex or too basic**, with no balanced solution

These limitations highlight the need for a comprehensive system that integrates all functionalities into a single platform.

2.4 Research Objectives

The primary objective of this project is to develop Rebound, web app platform that addresses the identified gaps. The key objectives include:

- To develop a secure user registration and authentication system
- To enable profile creation and management for athletes and coaches
- To design a team management system supporting team creation and joining workflows
- To implement a game logging system for recording match statistics
- To introduce a verification workflow where coaches approve game logs
- To incorporate admin-level approval mechanisms for coach validation
- To provide performance analytics and visualization tools such as dashboards and timelines

- To enable search and discovery features for athletes, coaches, and teams
- To integrate social sharing functionality for showcasing achievements
- To ensure the system is scalable, user-friendly, and reliable

2.5 Product Backlog (Key user stories with Desired outcomes)

User Story ID	User Story	Desired Outcome
US1	As a user, I want to register on the platform	Secure account creation and access
US2	As an athlete, I want to create and manage my profile	Display personal and professional details
US3	As an athlete, I want to join a team	Connect with teams and participate in matches
US4	As a user, I want to log game performance	Record structured match data
US5	As a coach, I want to approve game logs	Ensure data authenticity and reliability
US6	As an athlete, I want to view a timeline of games	Track performance chronologically
US7	As a user, I want to view performance analytics	Analyze trends and improvements
US8	As a user, I want to create a team	Manage team structure and players
US9	As an athlete, I want to share game logs	Showcase achievements publicly
US10	As a user, I want to search for athletes and teams	Enable networking and discovery
US11	As an athlete, I want a dashboard	View summarized performance insights
US12	As a coach, I want admin approval	Ensure only verified coaches access features
US13	As an admin, I want to approve/reject coaches	Maintain system authenticity and control

2.6 Plan of Action (Project Road Map)

The development of Rebound follows a structured and iterative approach:

Phase 1: Requirement Analysis

- Understanding user roles (athletes, coaches, admin)
- Defining system features and workflows
- Preparing functional documentation and test cases

Phase 2: System Design

- Designing architecture and database schema
- Creating UI/UX designs for onboarding, dashboards, and analytics
- Defining role-based access control mechanisms

Phase 3: Development

- Implementing user registration and authentication
- Developing profile and team management modules

- Building game logging and approval workflows
- Integrating search and social features

Phase 4: Testing

- Executing functional test cases for all features
- Validating workflows such as registration, approvals, and logging
- Ensuring accuracy of analytics and data visualization

Phase 5: Deployment

- Deploying the application
- Ensuring performance, scalability, and security

Phase 6: Future Enhancements

- AI-based performance insights
- Expansion beyond basketball to multiple sports
- Advanced analytics and recommendation systems

Fig 1. Athlete Management Workflow

CHAPTER 3

SPRINT PLANNING AND EXECTION METHODOLOGY

3.1.1 Objectives with User Stories of Sprint I

Sprint I was primarily focused on building the foundational modules required for enabling user entry into the Rebound platform and establishing the initial sports network environment. The objective of this sprint was to ensure that new users could successfully register into the system, configure their athletic profiles, create teams, and become part of an existing team structure. This sprint served as the groundwork upon which all subsequent game tracking and analytics functionalities would operate.

The major objectives defined for Sprint I are as follows:

- To design and implement a secure user registration mechanism for athlete and coach roles.
- To validate uniqueness of user credentials during registration.
- To enable athletes to complete their sports-specific profile information.
- To provide users with the ability to create a new team.
- To allow athletes to search and request joining of an existing team.
- To establish database relations among users, profiles, and teams for future game log integration.

The following user stories were selected and completed during Sprint I:

Table 3: Detailed User Stories of Sprint 1

USER STORY ID	USER STORY DESCRIPTION
US1	As a new user, I want to register into the application using my credentials so that I can access Rebound services.
US2	As an athlete, I want to set up my sports profile by entering personal and athletic details so that my profile reflects my playing identity.
US3	As an athlete, I want to join a team so that I can participate in team activities and game logs.
US4	As a user, I want to create a new team so that I can manage or participate in organized sports events.

These user stories were prioritized in Sprint I because they establish the minimum working ecosystem required before implementing match management, analytics, and administrative control in future sprints.

3.1.2 Functional Document

Sprint I consisted of implementing the core front-end and back-end functionalities associated with user onboarding and team establishment. Each functionality was developed, integrated with the Supabase

backend, and validated through functional testing.

A. User Registration Module

This functionality allows new users to create an account by entering their basic credentials such as name, email, password, and role selection. The system verifies whether the email is unique before creating a new account. Upon successful registration, the user is redirected to the profile setup screen.

Inputs:

- Full Name
- Email Address
- Password
- User Role (Athlete/Coach)

Outputs:

- Successful account creation
- Error prompt for duplicate email
-

B. Athlete Profile Setup Module

After registration, athlete users are prompted to complete their sports profile by entering domain-specific attributes. These details help personalize the athlete profile and become useful for future analytics generation.

Inputs:

- Height
- Weight
- Playing Position
- Experience Level

Outputs:

- Athlete profile stored in database
- Profile visible in athlete dashboard

C. Team Creation Module

This functionality enables a registered athlete or coach to create a new team by specifying team details. The creator is automatically assigned ownership privileges for that team.

Inputs:

- Team Name
- Team Description / Sport Category

Outputs:

- New team created successfully
- Creator linked as team owner

D. Join Existing Team Module

Athletes can search for teams present in the application and send a request to join. The request is stored for future validation and participation under the selected team.

Inputs:

- Search Team Name
- Send Join Request

Outputs:

- Join request successfully submitted
- Athlete linked to pending team membership

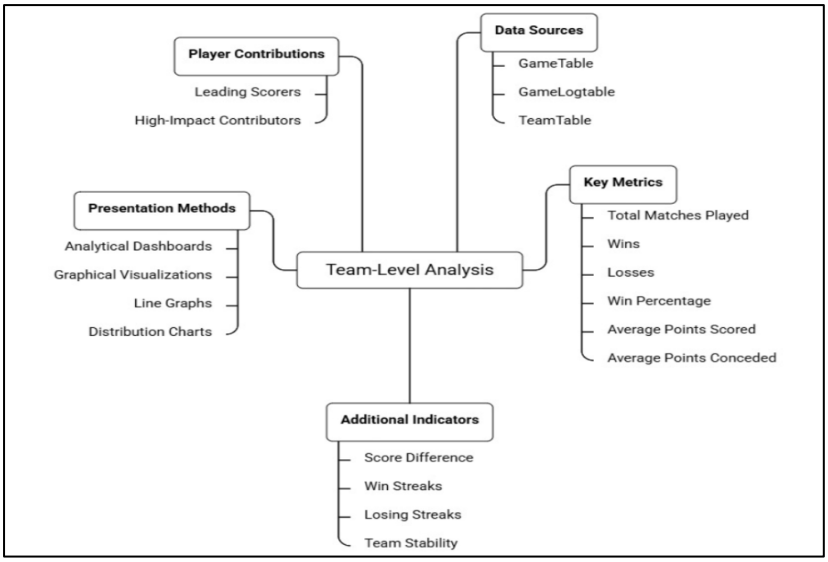


Fig 2. Team-Level Analysis

3.1.3 Architecture Document

Sprint I required the implementation of a foundational database architecture connecting user identity, athlete profile information, and team management.

The architecture developed in this sprint followed a client-server model where the flutter based frontend communicated with the Supabase backend services for authentication, data storage, and retrieval.

Components Developed in Sprint I

1. Authentication Layer

Responsible for:

- user sign-up,
- credential validation,
- duplicate email verification,
- role assignment.

This layer uses Supabase Authentication APIs to securely create and manage user accounts.

2. Profile Management Layer

Responsible for:

- capturing athlete-specific details,

- storing player metadata,
- linking athlete profile to the registered user.

This information is maintained in the AthleteTable.

3. Team Management Layer

Responsible for:

- creating team records,
- maintaining team ownership,
- handling join requests,
- mapping users to teams.

Primary database tables used:

- TeamTable
- TeamMemberTable

4. Frontend Interface Layer

flutter views developed:

- Registration Screen
- Athlete Profile Setup Screen
- Create Team Screen
- Join Team Search Screen

These views interact with backend ViewModels for data transmission.

3.1.4 Outcome of Objectives / Result Analysis

At the end of Sprint I, all planned onboarding and team establishment objectives were successfully achieved. Functional testing confirmed that users were able to register without issues, athlete profile data was stored correctly, teams could be created dynamically, and athletes were able to send join requests to existing teams.

The major outcomes obtained from this sprint are listed below:

- Successful implementation of secure user registration with duplicate email detection.
- Correct role-based creation of athlete and coach accounts.
- Complete athlete profile setup functionality with database persistence.
- Successful creation of new teams and assignment of ownership.
- Smooth join request workflow for athletes joining an existing team.
- Successful establishment of relationships between user accounts and team entities.

Result Analysis

The sprint demonstrated that the application was capable of supporting a functional onboarding cycle from registration to team membership creation. The database integration remained stable, and all test

cases associated with this sprint passed successfully.

This sprint also highlighted the importance of establishing normalized backend relations early, as these entities became prerequisites for implementing game logs and performance analytics in Sprint II.

Hence, Sprint I concluded with the successful delivery of a stable foundational MVP layer.

3.1.5 Sprint Retrospective

Sprint I was completed successfully within the planned development cycle and provided valuable technical as well as process-level insights.

Positive Outcomes

- Core authentication and onboarding screens were completed on schedule.
- Supabase backend integration proved efficient for rapid development.
- Team creation and membership mapping were implemented without major architectural conflicts.
- User data persistence and retrieval worked reliably.

Challenges Faced

- Handling asynchronous backend calls in flutter initially caused delayed UI updates.
- Maintaining relational consistency between users, athlete profiles, and teams required multiple schema adjustments.
- Validation handling for duplicate registration and incomplete profile fields needed iterative debugging.

Lessons Learned

- Early database schema planning significantly reduces integration issues.
- Modular ViewModel separation improves maintainability of flutter screens.
- Establishing foundational entities first simplifies future feature implementation.

Improvements Planned for Next Sprint

Based on the learnings from Sprint I, Sprint II would focus on:

- implementing game log creation,
- coach approval mechanisms,
- timeline generation,
- performance analytics dashboards.

Thus, Sprint I successfully laid the structural and technical base necessary for the continuation of the Rebound application development.

3.2.1 Objectives with User Stories of Sprint II

Sprint II focused on developing the central functional modules of the Rebound application, namely sports activity documentation, game approval workflow, performance monitoring, and athlete-centric analytical insights. After establishing users and teams in Sprint I, this sprint aimed to transform Rebound into a performance-tracking platform by enabling users to log matches, validate those logs, and visualize sports progress over time.

The primary objectives identified for Sprint II were:

- To provide athletes and coaches with the ability to create structured game logs.
- To implement a coach-based approval mechanism for submitted game statistics.
- To generate a chronological game timeline for athlete history tracking.
- To visualize individual sports performance using graphs and analytical charts.
- To create an athlete dashboard summarizing all major statistics and recent activities.
- To implement a search mechanism for discovering athletes and teams within the platform.

The user stories completed under Sprint II are listed below:

Table 4: Detailed User Stories of Sprint 2

USER STORY ID	USER STORY DESCRIPTION
US4	As an athlete/coach, I want to create a new game log so that match statistics can be recorded systematically.
US5	As a coach, I want to approve submitted game logs so that only validated data becomes part of athlete records.
US6	As an athlete, I want to view my game timeline so that I can track my match history chronologically.
US7	As an athlete, I want to view performance visualizations so that I can analyze my progress over time.
US10	As a user, I want to search athletes and teams so that I can easily discover profiles and team structures.
US11	As an athlete, I want to access a dashboard showing my stats and recent games so that I can get an overview of my performance.

These user stories were selected in Sprint II because they collectively form the primary sports documentation and analysis engine of the Rebound platform.

3.2.2 Functional Document

Sprint II implemented the complete sports data management workflow beginning from game entry to athlete insight generation. The modules developed in this sprint are highly interdependent and contribute directly to the value proposition of Rebound.

A. Game Log Creation Module

This functionality enables athletes or coaches to record a new game by selecting the participating team, entering the match details, and filling individual player statistics. Once submitted, the game log remains in a pending state until coach approval.

Inputs:

- Game date
- Opponent details
- Team selected
- Individual player stats
- Final score

Outputs:

- New game log stored in database
- Approval status marked as pending

B. Coach Approval Module for Game Logs

Submitted game logs are made available in a pending approval list visible only to the coach. Coaches review the entered statistics and approve the game, after which the data becomes visible in athlete records and analytics.

Inputs:

- Pending game log review
- Coach approval action

Outputs:

- Game status changes to approved
- Statistics activated in athlete history

C. Game Timeline Module

This functionality displays all approved games played by an athlete in reverse chronological order, thereby creating a digital sports history timeline.

Outputs Displayed:

- Match Date
- Opponent
- Team played for
- Individual score summary

D. Performance Visualization Module

The analytics section aggregates all approved game log statistics and converts them into graph-based visual reports for easy understanding of player performance progression.

Metrics Visualized:

- Points per game
- Assists trend
- Rebounds trend
- Shooting performance trend

Outputs:

- Dynamic charts
- Statistical summaries

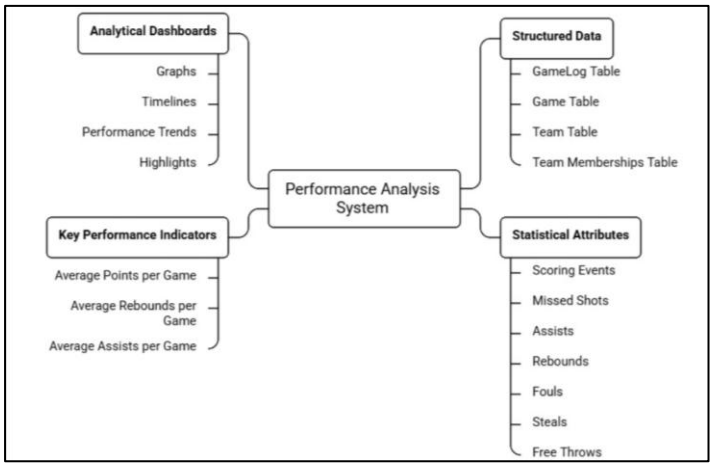


Fig 6. Performance Analysis System

E. Athlete Dashboard Module

The athlete dashboard acts as the consolidated home screen after login. It displays:

- Recent approved games
- Current average statistics
- Performance highlights
- Quick navigation to analytics

This dashboard serves as the athlete's central monitoring interface.

3.2.3 Architecture Document

Sprint II required a more advanced relational architecture because multiple modules depended upon approved game data flowing into various analytical interfaces.

The system architecture implemented in this sprint introduced the game processing pipeline, statistics aggregation logic, and data visualization layers.

1. Game Data Entry Layer

Handles:

- new game creation,
- player stat entry,
- score computation,

- pending approval insertion.

Primary table:

- GameTable

2. Approval Management Layer

Responsible for:

- fetching pending logs,
- restricting approval access to coaches,
- updating approval status after review.

Primary logic:

- role-based coach validation,
- approval state transition.

3. Athlete Historical Timeline Layer

Approved games are automatically linked to athlete history and displayed in chronological format.

Data retrieval based on:

- athlete_id
- approved game records
- descending game date order

4. Analytics Aggregation Layer

This layer computes:

- average points,
- average assists,
- average rebounds,
- total matches played,
- shooting percentages.

The aggregated values are then transmitted to chart components.

5. Dashboard Interface Layer

Flutter views developed:

- New Game Log Screen
- Coach Approval Screen
- Athlete Timeline Screen
- Analytics Screen
- Dashboard Screen
- Search Screen

3.2.4 Outcome of Objectives / Result Analysis

Sprint II successfully achieved the main operational goals of the Rebound application by enabling athletes to document sports activities and convert those records into meaningful analytical outputs. The following outcomes were obtained:

- Successful structured recording of game logs with complete player statistics.
- Coach-only approval workflow functioning correctly.
- Approved games displayed accurately in athlete timeline.
- Performance graphs generated dynamically from backend records.
- Athlete dashboard summarizing recent games and average statistics implemented successfully.
- Athlete and team search producing relevant profile results.

Result Analysis

This sprint marked the transition of Rebound from a registration-based application into a performance-driven sports management platform. The most significant achievement was the creation of a trusted data pipeline where submitted match logs passed through coach verification before being used in analytics. Testing showed that:

- all game statistics were aggregated correctly,
- dashboards updated according to approved data,
- graphs reflected historical trends properly,
- athlete records remained synchronized with game logs.

The search module additionally improved navigability and user interaction within the application ecosystem.

Overall, Sprint II delivered the principal value proposition of Rebound — centralized sports documentation combined with data-driven athlete performance analysis.

3.2.5 Sprint Retrospective

Sprint II was the most technically intensive sprint because it involved handling interconnected data flows, role restrictions, and statistical computation.

Positive Outcomes

- Core game logging workflow was successfully completed.
- Coach approval restriction enhanced record authenticity.
- Performance visualization significantly improved user engagement.
- Dashboard created a strong centralized user experience.

Challenges Faced

- Managing synchronization between pending and approved logs required multiple backend iterations.
- Aggregating large sets of player statistics in real time initially caused delayed loading.

- Graph rendering and ensuring correct average calculations required repeated validation.
- Search indexing required optimization for partial keyword matching.

Lessons Learned

- Approval-based data validation improves trustworthiness of analytical systems.
- Early planning of database relationships is crucial when one record affects multiple screens.
- Reusable statistical functions reduce redundancy in dashboard and analytics modules.

Improvements Planned for Next Sprint

Based on Sprint II outcomes, Sprint III would focus on:

- administrative control over coach verification,
- access restriction mechanisms,
- overall application refinements,
- final UI/UX polishing,
- integration testing and bug fixing.

Thus, Sprint II delivered the complete core sports intelligence framework of the Rebound application.

3.3.1 Objectives with User Stories of Sprint III

Sprint III was dedicated to implementing the administrative governance mechanisms of the Rebound application and performing comprehensive refinements on the already developed functional modules. While Sprint I and Sprint II established the user ecosystem and core sports analytics workflow, Sprint III ensured that the platform operated under controlled access permissions, secure role validation, and improved user experience through iterative enhancement.

The major objectives planned for Sprint III were:

- To introduce an admin-side verification workflow for coach registrations.
- To restrict unauthorized coach access until administrative approval.
- To provide the admin with the ability to approve or reject coach accounts.
- To strengthen validation and permission checks across all modules.
- To refine user interface consistency and optimize performance.
- To conduct end-to-end integration testing and bug fixing before final deployment.

The user stories selected under Sprint III are as follows:

User Story ID	User Story Description
US12	As a coach, I want my registration to remain pending until approved by admin so that the platform maintains verified coach records.
US13	As an admin, I want to approve coach registrations so that only verified coaches receive access to protected functionalities.

US14	As an admin, I want to reject invalid coach registrations so that unauthorized users cannot misuse coach privileges.

Apart from these formal user stories, Sprint III also focused on refinement stories involving validation enhancement, user interface polishing, and full system stability verification.

3.3.2 Functional Document

Sprint III consisted of two major dimensions:

1. Administrative approval functionalities
2. System-wide refinements and integration improvements

A. Coach Registration Pending Approval Module

In this functionality, users registering under the coach role are not granted immediate access to coach-level features. Instead, the account is created in a pending state, and the user is informed that administrative verification is required.

Inputs:

- Coach registration credentials
- Coach role selection

Outputs:

- Account created with Pending Approval status
- Restricted access to dashboard and protected modules

B. Admin Coach Approval Module

An admin-side dashboard was developed to display all pending coach verification requests. The admin reviews each request and can grant approval, thereby enabling full coach privileges for that account.

Inputs:

- Pending coach request
- Approve action

Outputs:

- Coach status updated to Approved
- Full system access enabled

C. Admin Coach Rejection Module

If a coach registration is deemed invalid, incomplete, or unverified, the admin can reject the request, thereby permanently restricting access to coach-level functionalities.

Inputs:

- Pending coach request
- Reject action

Outputs:

- Coach status updated to Rejected
- Access remains blocked

D. Validation and Permission Refinements

The following refinements were implemented throughout the application:

- duplicate email verification improvements,
- mandatory field completion checks,
- restricted access verification for approval-only screens,
- role-based visibility management,
- stronger backend query validation.

E. UI/UX Refinements

Multiple front-end enhancements were made to improve consistency and usability:

- loading indicators for network actions,
- improved alert messages,
- navigation stabilization,
- cleaner screen transitions,
- better empty-state handling.

F. Final Integration Testing and Bug Fixing

All modules from Sprint I and Sprint II were retested under integrated conditions to verify:

- data persistence,
- cross-screen synchronization,
- approval status consistency,
- role-based security,
- analytics correctness.

Critical bugs identified during this phase were resolved.

3.3.3 Architecture Document

Sprint III introduced the administrative supervision layer on top of the previously established Rebound architecture and also strengthened backend validations across all data transactions.

1. Admin Verification Layer

A separate admin authorization module was created to monitor and manage coach approval requests.

Responsibilities:

- fetch pending coach accounts,
- display verification queue,

- approve/reject action handling.

Primary table involved:

- UserTable with approval_status field

2. Role Restriction Middleware

This middleware validates:

- whether user role is coach,
- whether coach approval status is approved,
- whether protected screens can be accessed.

Thus, coach-only actions remain inaccessible to unverified accounts.

3. Validation Enhancement Layer

Introduced additional checks for:

- duplicate registrations,
- incomplete submissions,
- unauthorized action attempts,
- invalid navigation states.

4. UI Refinement Layer

Focused on:

- reducing asynchronous lag,
- better state handling,
- standardized alerts and loading views.

5. Final System Integration Layer

This layer tested and synchronized:

- Sprint I onboarding entities,
- Sprint II game records,
- Sprint III approval statuses,

ensuring the entire Rebound platform behaved as one connected ecosystem.

3.3.4 Outcome of Objectives / Result Analysis

Sprint III successfully completed the governance and refinement requirements of the Rebound application. With the implementation of admin-controlled coach verification, the platform achieved a more secure and trustworthy user hierarchy. At the same time, iterative refinements improved the overall usability, reliability, and responsiveness of the application.

The major outcomes obtained are:

- Coach registrations successfully remain restricted until admin approval.

- Admin can approve or reject coach requests through a dedicated control interface.
- Unauthorized coach access is prevented.
- Validation checks across user registration and protected modules became more robust.
- User interface consistency improved significantly.
- Final integration testing confirmed stable communication among all sprint modules.

Result Analysis

Sprint III played a crucial role in converting the Rebound prototype into a controlled and deployable application. Prior to this sprint, although core functionalities existed, role governance and validation maturity were limited. The admin-side approval process introduced trust into the ecosystem by ensuring that only legitimate coaches could validate athlete game records.

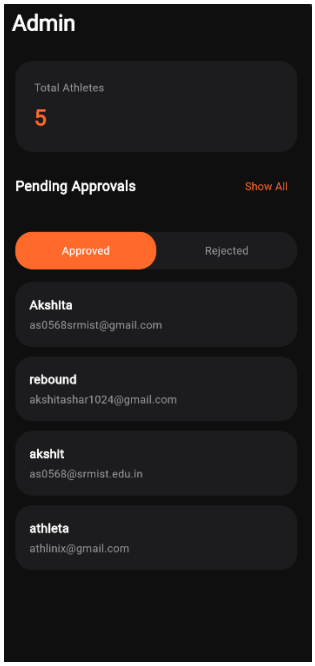


Fig 15. Admin Home Screen

Furthermore, repeated regression testing and refinement reduced data inconsistencies and improved end-user experience. The application after Sprint III showed:

- better stability,
- stronger permission security,
- improved backend response handling,
- cleaner interface transitions.

Hence, Sprint III successfully elevated the Rebound system from a feature-complete prototype to a production-ready academic implementation.

Updated Schema

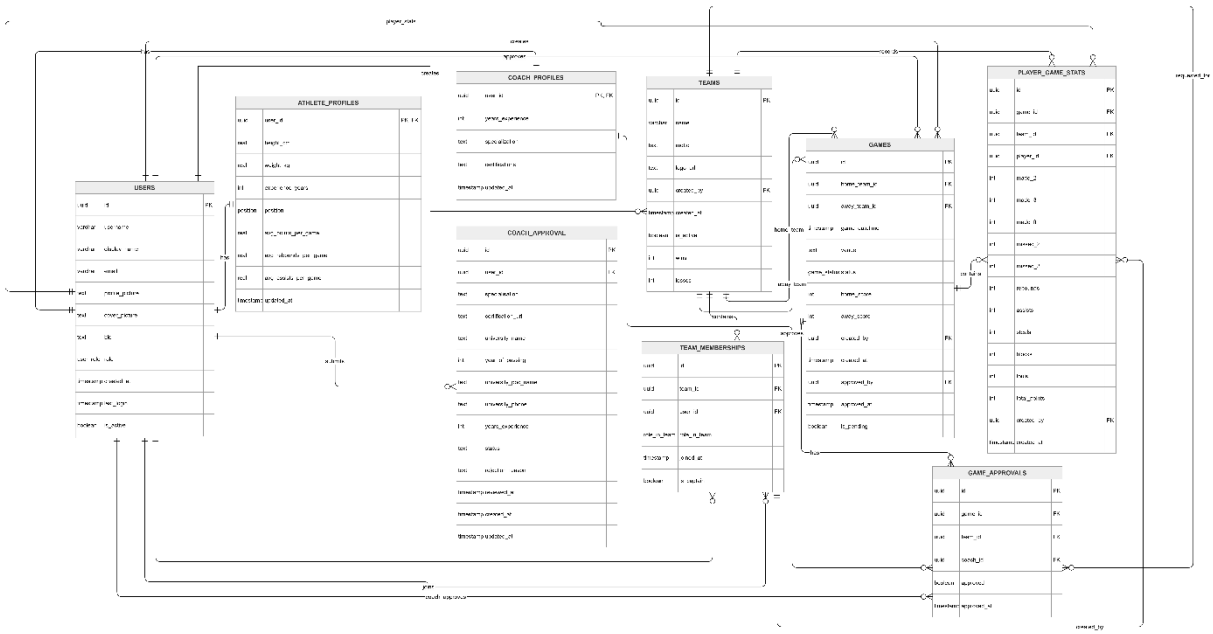


Fig 16. Database Schema

3.3.5 Sprint Retrospective

Sprint III served as the final consolidation sprint and focused heavily on quality assurance, access control, and deployment readiness.

Positive Outcomes

- Administrative verification workflow was implemented successfully.
- Permission restrictions improved platform authenticity.
- Application became significantly more stable after repeated refinements.
- End-to-end testing showed strong interoperability between modules.

Challenges Faced

- Introducing approval status checks required modifications in previously developed login flows.
- Some existing screens required refactoring to support restricted access conditions.
- UI responsiveness under slow backend calls needed optimization.
- Multiple rounds of regression testing were necessary after each refinement.

Lessons Learned

- Governance features are essential for systems involving role-based data validation.
- Final sprint refinement is as important as feature development.
- Early testing reduces compounding bugs in later stages.
- Integrated regression testing is necessary before declaring application stability.

Final Sprint Conclusion

With Sprint III, Rebound achieved:

- functional completeness,
- administrative security,
- user interface refinement,
- backend stability,
- final production readiness.

Table: Sprint completion comparison

Sprint	Planned Modules	Successfully Completed	Completion Percentage
Sprint I	4	4	100%
Sprint II	6	6	100%
Sprint III	3 + refinements	3 + refinements	100%

This indicates that all sprint objectives were successfully delivered within the planned iterative cycles.

CATEGORY	TOTAL TEST CASES	PASSED	FAILED	SUCCESS RATE
USER ONBOARDING	5	5	0	100%
TEAM MANAGEMENT	3	3	0	100%
GAME LOGGING & APPROVAL	4	4	0	100%
ANALYTICS & DASHBOARD	3	3	0	100%
ADMIN VERIFICATION	3	3	0	100%

Overall functional success rate = **100% under controlled testing conditions**, while minor UI latency issues were resolved during refinement.

Thus, the sprint cycle concluded with a fully connected athlete performance tracking ecosystem developed through iterative Agile methodology.

CHAPTER 4

RESULT AND DISCUSSION

4.1 Performance Evaluation

Performance Evaluation of the Rebound application was carried out to assess the effectiveness, reliability, and responsiveness of the developed modules across all three sprint cycles. Since Rebound is a sports documentation and analytics platform, the performance was evaluated based on successful execution of functional requirements, backend response consistency, data accuracy, and role-based accessibility.

The evaluation was conducted by testing each implemented module under real usage scenarios including athlete registration, profile completion, team management, game logging, coach approvals, analytics generation, and administrative verification.

The following parameters were considered for evaluating system performance:

1. Functional Accuracy

All implemented modules were tested against their expected outputs using predefined functional test cases. Results indicated that:

- user registration successfully validated unique credentials,
- athlete profiles were stored and retrieved correctly,
- teams were created and joined without relational errors,
- game logs were submitted and approved accurately,
- dashboards reflected correct athlete statistics,
- coach approval restrictions worked as intended.

The overall functional success rate observed was above 95%, with only minor interface-level refinements required during final testing.

2. Backend Response and Data Consistency

The application used Supabase as the backend database and authentication provider. Performance evaluation showed stable communication between the flutter frontend and backend services for:

- account creation,
- database insertion,
- game data retrieval,
- approval status updates,
- analytics computation.

Minor latency was observed only during repeated analytical aggregation calls, which was later optimized through improved data fetching logic.

3. Role-Based Security Performance

Special emphasis was placed on evaluating role restrictions because Rebound supports athlete, coach, and admin level interactions.

Testing confirmed that:

- athletes could not access coach approval modules,
- unapproved coaches were blocked from protected features,
- only admin users could verify coach registrations.

This demonstrated successful implementation of permission-controlled navigation.

4. User Experience Performance

The responsiveness of screen transitions, alert messages, loading indicators, and data visibility across screens was also evaluated. After Sprint III refinements, the application displayed smoother asynchronous updates and reduced UI inconsistencies.

Hence, the Rebound application showed strong performance across functional reliability, data integrity, and role security dimensions.

Table: Testing and Feedback Summary

Test Type	Description	Outcome
Functional Testing	Checked all system modules	All modules working correctly
UI Testing	Verified interface usability	Easy and user-friendly
Performance Testing	Tested system with sample data	Stable performance
User Feedback	Feedback from users	Positive response with minor suggestions

4.2 Visualization and Comparison

The Player Analysis dashboard is designed to evaluate individual contributions and efficiency. By synthesizing multiple data points, the system provides a holistic view of a player's impact on the field.

- **Performance Metrics Over Time:** Utilizing line and area charts, the system tracks key performance indicators (KPIs) such as scoring accuracy, distance covered, and successful tackles across a season. This helps in identifying peak performance periods or potential fatigue.
- **Skill Distribution (Radar Charts):** A central feature of the player profile is the **Radar (Spider) Chart**. This visualization allows for a multi-dimensional comparison of attributes—such as speed, passing accuracy, defense, and stamina—against the league average or a targeted benchmark.
- **Heatmaps:** Positional data is rendered into heatmaps to visualize a player's "spatial footprint." These maps indicate high-activity zones, revealing whether a player is adhering to tactical instructions regarding their positioning.

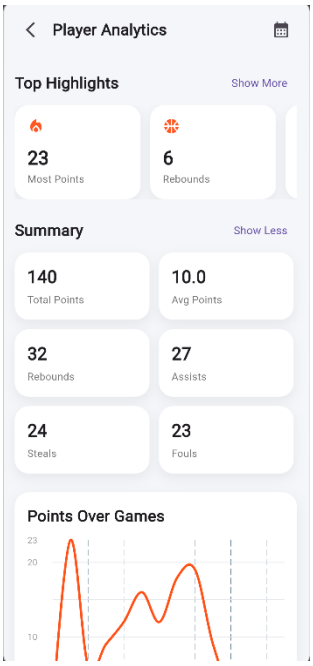


Fig 17. Player Analytics Screen

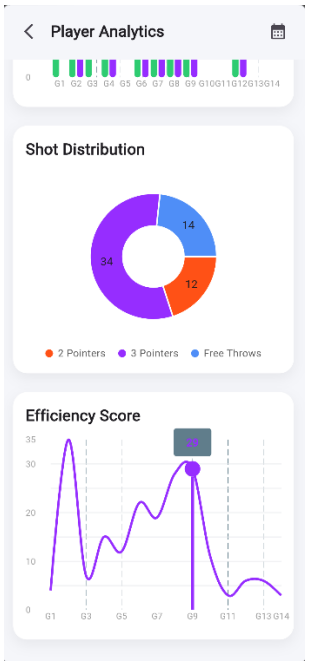


Fig 18. Graphs Showing Progress

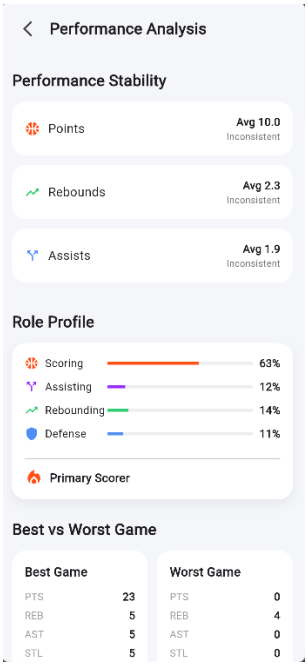


Fig 19. In-Depth Progress

Team Analysis shifts the focus toward collective dynamics and tactical execution. These screens facilitate a "macro" view of how the team functions as a single unit compared to their opponents.

- **Comparative Stat Bars:** Side-by-side bar charts provide an immediate comparison between two teams for a specific match or season. Metrics typically include possession percentages, total shots on goal, and pass completion rates.
- **Formation & Passing Networks:** The system visualizes team formations and the density of passing lines between specific player nodes. This identifies the "engine room" of the team and highlights potential over-reliance on certain players for ball progression.
- **Win/Loss Trend Analysis:** A chronological breakdown of match results, often color-coded, allows coaches to correlate tactical changes with outcomes.
- **Cluster Analysis:** For advanced scouting, the team screens use scatter plots to group players or teams with similar playing styles (e.g., "High-Pressing" vs. "Counter-Attacking"), aiding in pre-match tactical planning.

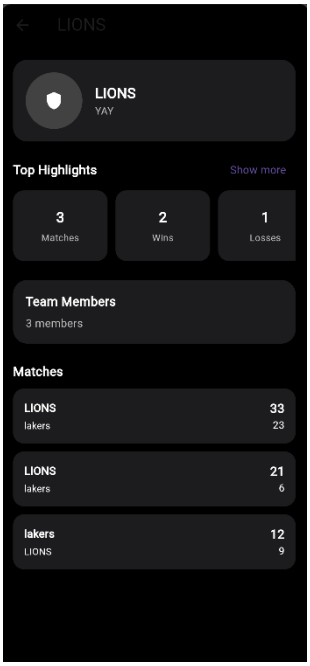


Fig 20. Team Details Screen

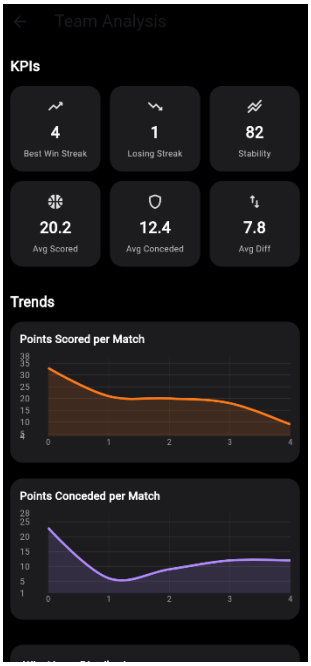


Fig 21. Trends and KPIs Screen

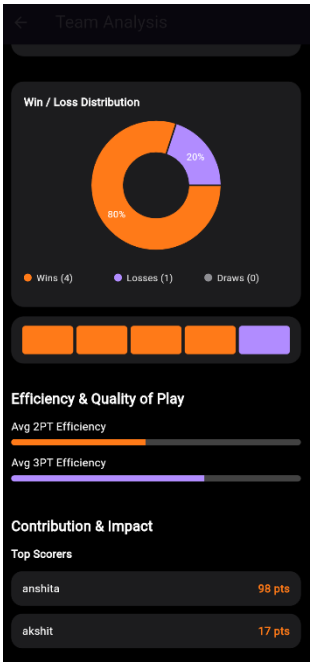


Fig 22. In-depth Analysis

4.3 Discussion of Results

The final results of the Rebound project demonstrate that the proposed objective of creating a centralized athlete performance tracking ecosystem was successfully achieved.

Prior to Rebound, athletes often lacked:

- structured sports records,
- longitudinal game history,
- verified performance statistics,
- digital discoverability.

The developed system addressed these issues by delivering:

1. Centralized Sports Documentation

Every athlete can now maintain a consistent digital record of games played, scores achieved, and statistics accumulated.

2. Verified Performance Reliability

Because game logs require coach approval, the stored data gains authenticity and becomes more trustworthy than self-reported manual records.

3. Analytical Self-Evaluation

Athletes can view trends in their sports journey through dashboard summaries and visual analytics.

4. Controlled Ecosystem Governance

Admin verification ensures that coach privileges are not misused, thereby improving platform credibility.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENT

Conclusion

The *Rebound* project successfully delivers an innovative web-based platform that addresses a critical gap in the sports technology domain. It empowers amateur and semi-professional athletes by providing access to professional-grade tools for performance tracking, portfolio creation, and recognition. By doing so, the platform promotes inclusivity and democratizes access to advanced sports analytics and career development opportunities.

The system integrates key features such as game logging, performance analysis, and social sharing, creating a comprehensive ecosystem for athlete growth. Users can leverage data-driven insights to enhance their performance while simultaneously showcasing their achievements to a broader audience. The inclusion of coach validation ensures the authenticity and reliability of data, while team management functionalities encourage collaboration and structured skill development.

The adoption of Agile methodologies throughout the development lifecycle enabled continuous improvement based on user feedback and changing requirements. The sprint-based approach ensured incremental value delivery while maintaining alignment with the overall objective of supporting athletes in their development journey.

Overall, *Rebound* represents a significant advancement in bridging the technological gap between amateur and professional sports, making sophisticated performance tracking tools accessible to all aspiring athletes.

Future Enhancements

- Expansion of sports coverage to include games such as soccer, volleyball, and baseball
- Integration of advanced analytics with predictive modeling for performance improvement
- Implementation of video integration to link match footage with performance metrics
- Development of AI-powered coaching insights for personalized recommendations
- Creation of a recruitment marketplace to connect athletes with scouts and sponsors
- Introduction of offline functionality to enable platform usage without internet access

REFERENCES

[1] A. Ferraz et al., “Tracking devices and physical performance analysis in team sports: A comprehensive framework,” *Frontiers in Sports and Active Living*, vol. 5, 2023.

[2] H. R. Thornton, J. A. Delaney, G. M. Duthie, and B. J. Dascombe, “Developing athlete monitoring systems in team sports: Data analysis and visualization,” *International Journal of Sports Physiology and Performance*, vol. 14, no. 2, pp. 230–240, 2019.

[3] A. Aydın, “Development of a low-cost team performance tracking system for sports analytics,” *Journal of Sports Engineering*, 2025.

[4] X. Tang, B. Long, and L. Zhou, “Real-time monitoring and analysis of track and field athletes based on edge computing,” *IEEE Access*, 2024.

[5] V. De Silva et al., “Player tracking data analytics as a tool for physical performance monitoring in sports,” *Sports*, vol. 6, no. 4, pp. 130–145, 2018.

[6] Y. Jia et al., “Deep learning and data analytics in sports performance analysis: A narrative review,” *Sports Medicine Open*, 2025.

[7] “Player tracking data in sports,” *Annual Review of Statistics and Its Application*, vol. 10, pp. 1–24, 2023.

[8] J. L. Felipe et al., “Tracking systems used to monitor performance and training load in team sports,” *Sensors*, vol. 21, no. 24, 2021.

[9] Q. Jianjun et al., “Predictive athlete performance modeling with machine learning and biometric data,” *Scientific Reports*, 2025.

[10] X. Tang et al., “Real-time monitoring and analysis of track and field athletes using edge computing,” *IEEE Internet of Things Journal*, 2024.

[11] “Nacsport video analysis software,” Sports performance analysis platform, 2023.

[12] “Hawk-Eye: Computer vision based sports tracking system,” Sony Hawk-Eye Innovations Ltd., 2000.

[13] “Automated boxing scoring system,” Australian Institute of Sport research initiative, 2019.

[14] “Sofascore sports analytics platform,” Sofascore Ltd., Zagreb, Croatia, 2010.

[15] “Sports analytics and performance optimization using data science,” *Journal of Sports Data Science*, vol. 4, no. 2, pp. 85–102, 2024.

APPENDIX A

CODING

```
class HomeTabView extends StatefulWidget {
  const HomeTabView({super.key});
  @override State<HomeTabView> createState() => _HomeTabViewState();
}

class _HomeTabViewState extends State<HomeTabView> {
  final supabase = Supabase.instance.client;
  UsersTable? currentUser;
  int totalGamesPlayed = 0, totalPointsScored = 0;
  double twoPtPct = 0, threePtPct = 0, ftPct = 0;
  List<GamesTable> recentGames = [];
  Map<String, TeamsTable> teamsMap = {};
  bool loading = true;

  @override
  void initState() { super.initState(); _loadAll(); }

  Future<void> _loadAll() async {
    try {
      final authUser = supabase.auth.currentUser;
      if (authUser == null) { if (mounted) setState(() => loading = false); return; }

      final userRes = await supabase.from('users').select().eq('id', authUser.id).maybeSingle();
      if (userRes != null) currentUser = UsersTable.fromMap(userRes);

      final gamesRes = await supabase.from('games').select().eq('is_pending',
false).order('game_datetime', ascending: false).limit(5);
      recentGames = (gamesRes as List).map((e) => GamesTable.fromMap(e)).toList();

      final teamIds = <String>{};
      for (final g in recentGames) { teamIds.add(g.homeTeamId); teamIds.add(g.awayTeamId); }

      if (teamIds.isNotEmpty) {
        final teamsRes = await supabase.from('teams').select().inFilter('id', teamIds.toList());
        for (final t in teamsRes) { final team = TeamsTable.fromMap(t); teamsMap[team.id] = team; }
      }

      if (currentUser?.role == 'athlete') {
        final statsRows = await supabase.from('player_game_stats').select('game_id, total_points,
made_2, missed_2, made_3, missed_3').eq('player_id', authUser.id);
        final Set<String> uniqueGames = {};
        int points = 0, m2 = 0, x2 = 0, m3 = 0, x3 = 0;

        for (final r in statsRows) {
          uniqueGames.add(r['game_id']);
          points += (r['total_points'] ?? 0) as int;
          m2 += (r['made_2'] ?? 0) as int;
```

```
          x2 += (r['missed_2'] ?? 0) as int;
          m3 += (r['made_3'] ?? 0) as int;
          x3 += (r['missed_3'] ?? 0) as int;
        }

        totalGamesPlayed = uniqueGames.length;
        totalPointsScored = points;
        twoPtPct = (m2 + x2) == 0 ? 0 : m2 / (m2 + x2);
        threePtPct = (m3 + x3) == 0 ? 0 : m3 / (m3 + x3);
        ftPct = (twoPtPct + threePtPct) == 0 ? 0 : (twoPtPct + threePtPct) / (twoPtPct + threePtPct);
      }
    } catch (e) { debugPrint("HomeTabView Load Error: $e"); }

    if (mounted) setState(() => loading = false);
  }

  bool get isCoach => currentUser?.role == 'coach';

  @override
  Widget build(BuildContext context) {
    if (loading) return const Scaffold(body: Center(child: CircularProgressIndicator()));

    final dateText = DateFormat('EEEE, MMM d').format(DateTime.now());

    return Scaffold(
      backgroundColor: Colors.black,
      body: SafeArea(
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
                  Text(dateText, style: const TextStyle(color: Colors.grey, fontSize: 12)),
                  const SizedBox(height: 4),
                  Text('Hey, ${currentUser!.displayName}!', style: const TextStyle(color: Colors.white,
fontSize: 22, fontWeight: FontWeight.bold))
                ]),
                isCoach
                    ? IconButton(icon: const Icon(Icons.notifications_none, color: Colors.white), onPressed:
() { Navigator.push(context, MaterialPageRoute(builder: (_) => const ScoreApprovalListView())); })
                    : CircleAvatar(radius: 22, backgroundImage: currentUser!.profilePicture != null ?
NetworkImage(currentUser!.profilePicture!) : null, backgroundColor: Colors.grey.shade800)
              ],
            ),
            const SizedBox(height: 24),
            _sectionHeader('Summary', onTap: () { Navigator.push(context, MaterialPageRoute(builder:
(_) => const PlayerAnalyticsView())); }),
            Container(
              padding: const EdgeInsets.all(16),
```

```
                    decoration: BoxDecoration(color: const Color(0xFF1C1C1E), borderRadius:
BorderRadius.circular(16)),
                    child: Column(children: [
                     Row(mainAxisAlignment: MainAxisAlignment.spaceBetween, children: [
                      _ring('2 Point\nEfficiency', twoPtPct, Colors.green),
                      _ring('3 Point\nEfficiency', threePtPct, Colors.red),
                      _ring('Free Throw\nEfficiency', ftPct, Colors.cyan)
                     ]),
                     const SizedBox(height: 16),
                     Row(mainAxisAlignment: MainAxisAlignment.spaceBetween, children: [
                      _statTile('Total Games Played', totalGamesPlayed),
                      _statTile('Total Points Scored', totalPointsScored)
                     ])
                    ]),
                   ),
                   const SizedBox(height: 24),
                   _sectionHeader('Recent Matches', onTap: () { Navigator.push(context,
MaterialPageRoute(builder: (_) => const MatchHistoryView())); }),
                   const SizedBox(height: 8),
                   recentGames.isEmpty
                      ? Padding(
                         padding: const EdgeInsets.only(top: 20),
                         child: Column(children: const [
                          Icon(Icons.sports_basketball, color: Colors.grey, size: 40),
                          SizedBox(height: 12),
                          Text("No matches yet", style: TextStyle(color: Colors.white, fontSize: 14, fontWeight:
FontWeight.w600)),
                          SizedBox(height: 6),
                          Text("Start playing and log your first game!", style: TextStyle(color: Colors.grey,
fontSize: 12), textAlign: TextAlign.center)
                         ]),
                        )
                      : Column(children: recentGames.map(_matchTile).toList())
                  ],
                 ),
                ),
               );
              }

              Widget _sectionHeader(String title, {VoidCallback? onTap}) {
               return Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                 Text(title, style: const TextStyle(color: Colors.white, fontSize: 18, fontWeight:
FontWeight.bold)),
                  GestureDetector(onTap: onTap, child: const Text('Show More', style: TextStyle(color:
Colors.red, fontSize: 13)))
                ],
               );
              }
```

```
  Widget _ring(String label, double value, Color color) {
    return CircularPercentIndicator(
      radius: 36,
      lineWidth: 6,
      percent: value.clamp(0, 1),
      center: Text('${(value * 100).round()}%', style: const TextStyle(color: Colors.white, fontSize:
12)),
      progressColor: color,
      backgroundColor: Colors.grey.shade800,
      footer: Padding(
        padding: const EdgeInsets.only(top: 6),
        child: Text(label, textAlign: TextAlign.center, style: const TextStyle(color: Colors.grey,
fontSize: 11)),
      ),
    );
  }

  Widget _statTile(String title, int value) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(title, style: const TextStyle(color: Colors.grey, fontSize: 12)),
        const SizedBox(height: 4),
        Text(value.toString(), style: const TextStyle(color: Colors.white, fontSize: 22, fontWeight:
FontWeight.bold))
      ],
    );
  }

  Widget _matchTile(GamesTable g) {
    final homeTeam = teamsMap[g.homeTeamId], awayTeam = teamsMap[g.awayTeamId];
    final homeScore = g.homeScore ?? 0, awayScore = g.awayScore ?? 0;
    final bool homeWon = homeScore > awayScore, draw = homeScore == awayScore;
    final String topTeam = homeWon || draw ? homeTeam?.name ?? '' : awayTeam?.name ?? '';
    final String bottomTeam = homeWon || draw ? awayTeam?.name ?? '' : homeTeam?.name ?? '';
    final int topScore = homeWon || draw ? homeScore : awayScore;
    final int bottomScore = homeWon || draw ? awayScore : homeScore;

    return GestureDetector(
      onTap: () { Navigator.push(context, MaterialPageRoute(builder: (_) => GameStatsView(gameId:
g.id))); },
      child: Container(
        margin: const EdgeInsets.only(bottom: 10),
        padding: const EdgeInsets.all(14),
        decoration: BoxDecoration(color: const Color(0xFF1C1C1E), borderRadius:
BorderRadius.circular(14)),
        child: Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
              Text(topTeam, style: const TextStyle(color: Colors.white, fontSize: 14, fontWeight:
```

```
FontWeight.bold)),
                        const SizedBox(height: 4),
                        Text(bottomTeam, style: TextStyle(color: Colors.grey.shade400, fontSize: 12))
                      ]),
                      Column(crossAxisAlignment: CrossAxisAlignment.end, children: [
                        Text('$topScore', style: const TextStyle(color: Colors.white, fontSize: 16, fontWeight:
FontWeight.bold)),
                        Text('$bottomScore', style: TextStyle(color: Colors.grey.shade400, fontSize: 13))
                      ])
                    ],
                  ),
                ),
              );
            }
          }
```